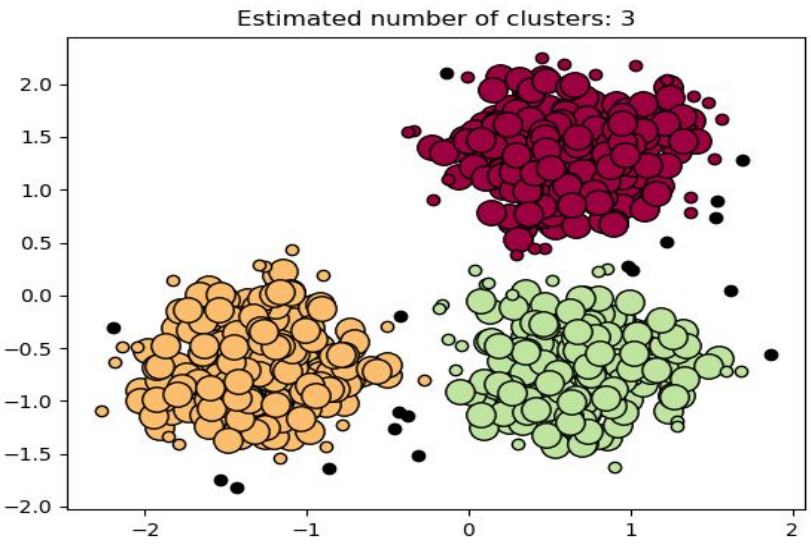
CLUSTERING

1) DBSCAN:

Group data points based on density.



Important Parameters:

1) **Epsilon:**

   Distance between 2 points

2) **Minpts:**

   Minimum no of points required to form a cluster.

Points in DBSCAN:

Core Point:

Points that can form the cluster.

Border Point:

Not enough to form a core point, but near to the core point.

Noise Point:

Individual point not close to cluster.

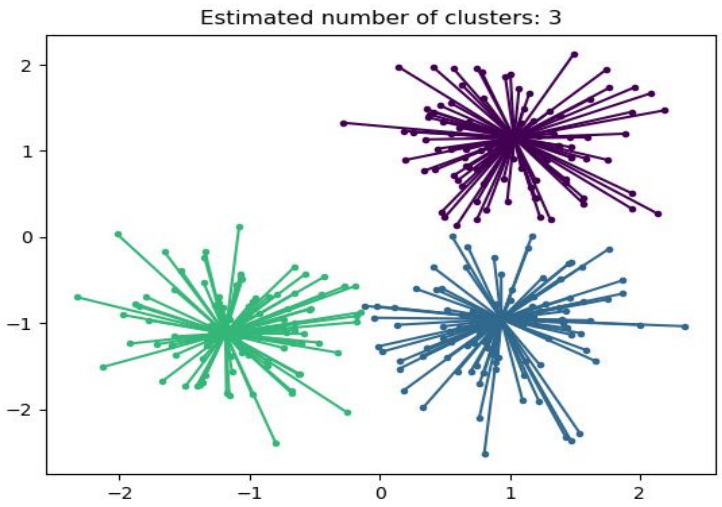
CODE:

```
from sklearn.cluster import DBSCAN

from sklearn.preprocessing import StandardScaler


scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)


model = DBSCAN(eps=0.5, min_samples=5)


labels = model.fit_predict(X_scaled)
```

2) Affinity Propagation:

Group data by sending message between the data points.

Exemplar is the data point that represent a cluster center.



Important Messages

1) **Responsibility ®:**

   It compares similarity between points.

2) **Availability (a):**

   This depends on how many other points need K as their center.

Important Parameter:

High Preferences:

More clusters

Low Preferences:

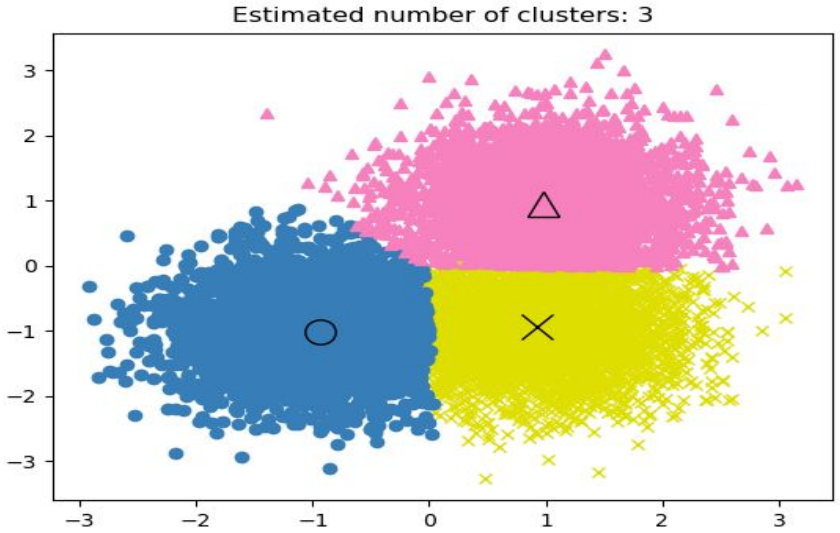
Fewer clusters

CODE:

```
from sklearn.cluster import AffinityPropagation

from sklearn.datasets import make_blobs

X, _ = make_blobs(n_samples=300, centers=4, random_state=42)

model = AffinityPropagation()

bbbb]p.,,,,,,,,,,,ē……Kmodel.fit(X)


# cluster labels

labels = model.labels_


# cluster centers

centers = model.cluster_centers_
```

3) MEAN SHIFT:

Mean Shift is a clustering algorithm that **iteratively shifts data points toward the mean of nearby points to locate dense regions and form clusters.**



Important Parameter

Bandwidth

This is the **radius of the neighborhood**.

Code: MeanShift(bandwidth=2)

- Small bandwidth → many clusters
- Large bandwidth → fewer clusters

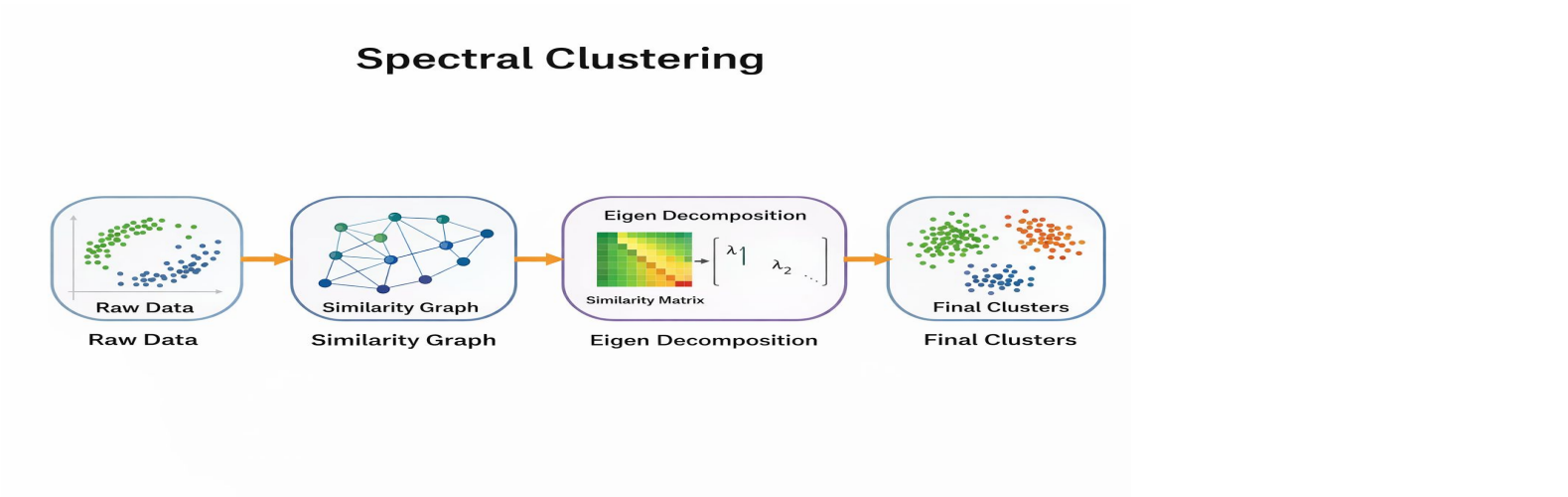
CODE

```
from sklearn.cluster import MeanShift

model = MeanShift()

cluster = model.fit_predict(X_scaled)
```

4) Spectral clustering

Spectral Clustering is an **unsupervised clustering algorithm** that groups data points using **graph theory and eigenvalues**. It is especially useful when clusters have **complex shapes** that algorithms like **K-Means** cannot detect.

Data → Similarity Graph → Eigenvector transformation → KMeans clustering



Important Parameter:

1. n_clusters

Number of clusters to create.

2. affinity

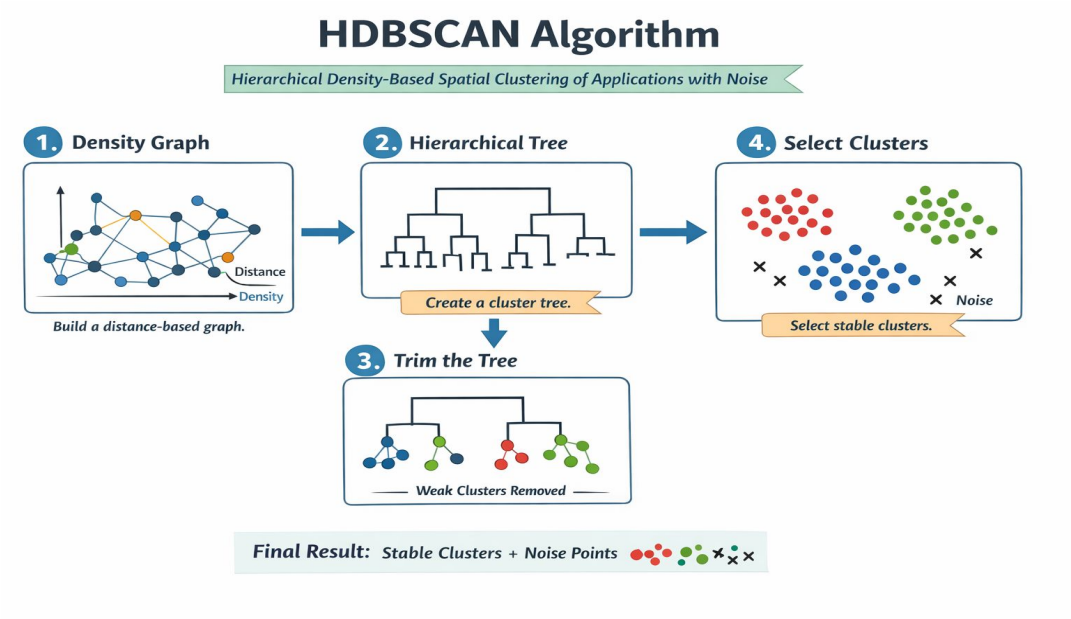
How similarity between points is calculated.

Code:

```
from sklearn.cluster import SpectralClustering

model = SpectralClustering(
n_clusters=5,
affinity='rbf',
random_state=5
)

cluster = model.fit_predict(X_scaled)
```

5) HDBSCAN

HDBSCAN stands for **Hierarchical Density-Based Spatial Clustering of Applications with Noise**. It is an improved version of DBSCAN used in **unsupervised learning** to find clusters based on **data density**.



Why HDBSCAN is used

Normal DBSCAN has one big limitation:

- It needs a fixed **epsilon (ε)** distance.
- If clusters have **different densities**, DBSCAN may fail.

HDBSCAN solves this problem by:

- Building a **hierarchy of clusters**
- Automatically selecting the **best clusters**
- Handling **clusters with different densities**

Important Parameter

Min_cluster_size

Minimum number of points required to form a cluster

Min_samples

Controls how strict the clustering is

Metric

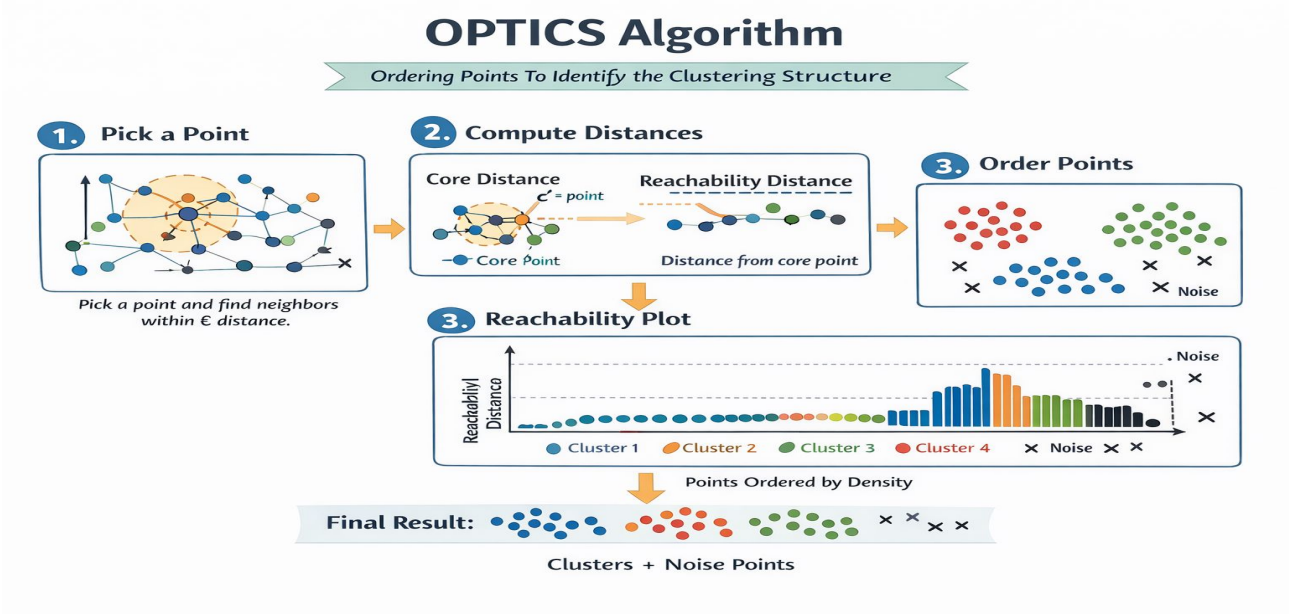
Distance measure (euclidean, manhattan etc.)

Code

```
import hdbscan


clusterer = hdbscan.HDBSCAN(min_cluster_size=10)

labels = clusterer.fit_predict(X_scaled)
```

6) OPTICS

OPTICS stands for **Ordering Points To Identify the Clustering Structure**. It is a **density-based clustering algorithm** similar to DBSCAN but designed to **handle clusters with different densities better**.



Why OPTICS is used

In DBSCAN, you must choose a fixed **epsilon (ε)** value.

Problem:

- If **ε is too small** → many points become noise
- If **ε is too large** → clusters merge incorrectly

OPTICS solves this problem by:

- Not fixing a single epsilon
- Creating an **ordering of points based on density**

So it can discover **clusters of varying density**.

Important Terms

Core Distance

Minimum distance required for a point to become a **core point**.

Reachability Distance

Distance that shows **how reachable one point is from another**.

Smaller reachability distance → higher density cluster.

Parameters

Min_samples

Minimum points required to form dense region

Max_eps

Maximum distance between points

Metric

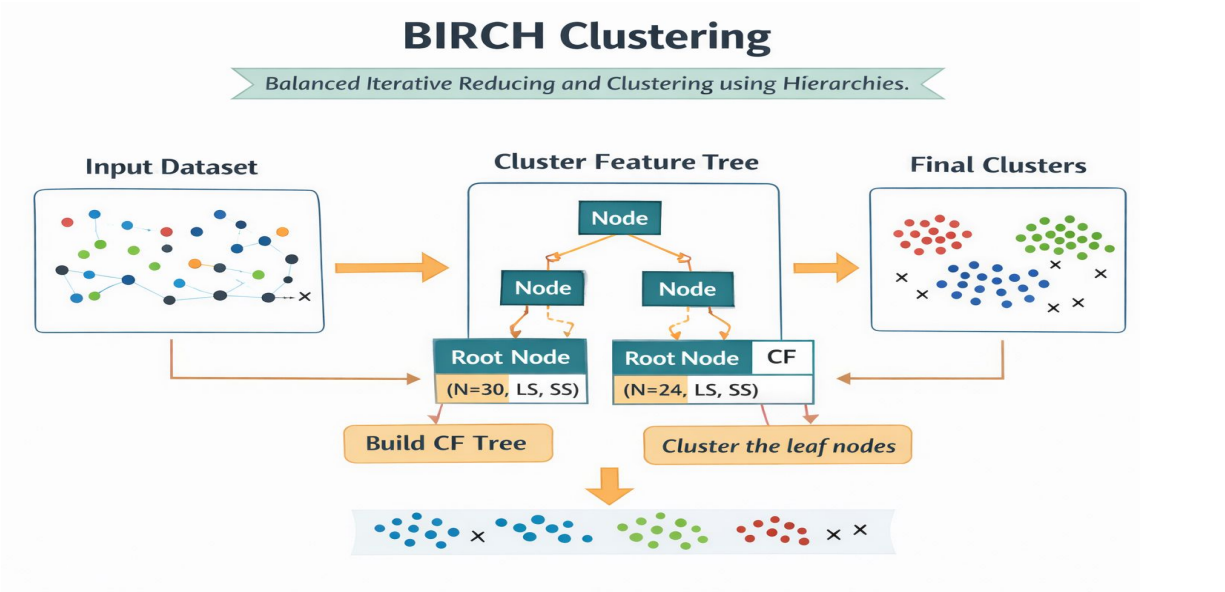
Distance method (euclidean etc.)

Code

```
from sklearn.cluster import OPTICS

model = OPTICS(min_samples=10)

labels = model.fit_predict(X)
```

7) BIRCH

BIRCH stands for **Balanced Iterative Reducing and Clustering using Hierarchies**. It is a **hierarchical clustering algorithm** designed to work efficiently with **very large datasets**.



Why BIRCH

Some clustering algorithms like **K-Means** or **Agglomerative Clustering** become slow when the dataset is very large.

BIRCH solves this problem by:

- Compressing data into a **compact structure**
- Clustering the compressed data instead of the full dataset

So it is **very memory efficient and fast**.

Basic Idea

BIRCH builds a special tree structure called a **CF Tree**.

CF = Clustering Feature

Each node stores summarized information about a cluster instead of storing all points.

The algorithm works in two main stages:

1. Build **CF Tree**

2. Perform clustering on leaf nodes

Clustering Feature (CF)

Each cluster is represented by:

- **N** → number of points
- **LS** → linear sum of points
- **SS** → square sum of points

Steps in BIRCH Algorithm

Step 1 — Scan dataset

The algorithm reads the dataset **one point at a time**.

Step 2 — Insert into CF Tree

Each point is added to the **closest cluster node**.

Step 3 — Apply threshold

If the cluster diameter exceeds the threshold, a **new cluster node is created**.

Step 4 — Final clustering

Optional clustering (like K-Means) can be applied on leaf nodes.

Important Parameters

Threshold

Maximum cluster diameter

Branching_factor

Maximum children per node

N_clusters

Number of final clusters

Code

```
from sklearn.cluster import Birch

model = Birch(threshold=0.5, n_clusters=4)

labels = model.fit_predict(X)
```